A Convolutional Neural Network With Mapping Layers for Hyperspectral Image Classification

Rui Li, Zhibin Pan, Yang Wang, and Ping Wang

***Abstract*— In this article, we propose a convolutional neural network with mapping layers (MCNN) for hyperspectral image (HSI) classification. The proposed mapping layers map the input patch into a low-dimensional subspace by multilinear algebra. We use our mapping layers to reduce the spectral and spatial redundancies and maintain most energy of the input. The feature extracted by our mapping layers can also reduce the number of following convolutional layers for feature extraction. Our MCNN architecture avoids the declining accuracy with increasing layers phenomenon of deep learning models for HSI classification and also saves the training time for its effective mapping layers. Furthermore, we impose the 3-D convolutional kernel on the convolutional layer to extract the spectral–spatial features for HSI. We tested our MCNN on three data sets of Indian Pines, University of Pavia, and Salinas, and we achieved the classification accuracy of 98.3%, 99.5%, and 99.3%, respectively. Experimental results demonstrate that the proposed MCNN can significantly improve classification accuracy and save much time consumption.**

***Index Terms*— Convolutional neural network (CNN), dimension reduction, feature extraction, hyperspectral image (HSI) classification, mapping layers.**

I. Introduction

HYPERSPECTRAL image (HSI) has drawn much attention in recent years. HSI processing is widely used in various remote sensing fields, such as land use analysis, urban planning, and environment monitoring [1]–[3]. In order to alleviate the high spectral dimension in HSI classification, many dimension reduction methods are proposed. Generally, they can be divided into two categories: feature selection methods and feature extraction methods. The feature selection methods aim at preserving the most representative bands and discarding those making no contributions to the classification results. Wang *et al.* [4] adopts manifold ranking as an unsupervised feature selection method to choose the most representative bands. Du and Yang [5] propose a similarity-based unsupervised band selection method. Moreover, the multitask joint sparse representation-based method [6] integrates the band selection method with the Markov random field. However, feature selection methods can only select existing bands from HSI. Thus, the feature extraction methods are designed to find the best method to map the original high-dimensional feature into a low-dimensional subspace while keeping the most useful dimensions for classification. The most representative methods are principal component analysis (PCA) [7] and its variants [8]–[10]. The linear discriminant analysis is also a widely used feature extraction method that searches the optimal projection subspace by maximizing between-class scatter matrix and minimizing within-class scatter matrix [11]. Apart from this, some graph-based feature extraction methods are proposed for its similarity to model relations among the objects [12], [13]. In this article, we focus on the feature extraction method in order to improve the HSI classification performance.

The complex spatial and spectral distributions make it difficult to classify the objects in HSI; a good methodology to incorporate the spatial and spectral features for HSI classification is the most concerned issue in this field. In Wang's locality and structure regularized low-rank representation (LSLRR) model [14], a new distance metric is introduced to combine both spatial and spectral features. However, the spectral bands in HSI are much more redundant than its spatial information, and thus, many traditional methods extract the most discriminative spatial features or bands and use them to train the classifiers. The support vector machine (SVM) is the most commonly used classifier in these traditional HSI classification methods. SVM-based classifiers are state-of-the-art methods with excellent classification accuracy for a long time. Recently, SVM is combined with edge-preserving filter to derive the spectral–spatial information for classification [15]. Not only suffering from curse of dimensionality, HSI classification also subjects to the problem of large spatial variability of spectral signature [16]. To deal with the spatial variability of spectral signature, the spectral information is extracted for classification in many methods, such as extended morphological profiles [17], spectral–spatial kernels [18], and superpixels [19]. In this article, we also try to extract spectral–spatial feature for HSI classification.

For many HSI classification methods, they have a drawback that one or two layers of nonlinear transformation are not enough to represent the spatial and spectral features. The linear

Manuscript received July 31, 2019; accepted September 28, 2019. Date of publication November 12, 2019; date of current version April 22, 2020. This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFC1504202 and Grant 2018YFC0603501 and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LQY19F010001. *(Corresponding author: Zhibin Pan.)*

R. Li is with the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an 710049, China.

Z. Pan and P. Wang are with the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an 710049, China, and also with the Research Institute, Xi'an Jiaotong University, Hangzhou 311215, China (e-mail: zbpan@mail.xjtu.edu.cn).

Y. Wang was with the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an 710049, China. He is now with the Department of Information Science, Xi'an University of Technology, Xi'an 710048, China.

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TGRS.2019.2948865

0196-2892 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

SVM and logistic regression can be viewed as a single-layer classifier, whereas decision tree and SVM with kernels are considered to have two layers [20]. It is known that human brains perform well in tasks, such as object recognition, for its multiple stages of processing from retina to cortex [21]. In order to extract the better invariant feature of data, a classifier of multilayer structure is advised. Therefore, deep architectures have been shown to yield promising performance in image classification task. Recently, with the development of deep learning, Chen *et al.* [22] first introduce deep learning to HSI classification. After that, many other deep learning methods are applied in HSI classification to improve the performance [23]–[25]. Wang *et al.* [26] employ recurrent convolutional network for scene classification of remote sensing. Among them, the convolutional neural network (CNN) has achieved promising performance in many HSI processing fields, and many researchers have proved that CNN can deliver state-of-the-art performance in HSI classification [27]. Deep learning techniques are able to automatically learn hierarchical features (from low level to high level) from raw input data, and the reason is that scene classification can explore the salient features in remote sensing scenes. Such learned features have achieved tremendous success in many machine vision tasks. Although deep learning methods, such as stacked autoencoder (SAE) [28] and deep brief network (DBN) [29], can extract the spectral–spatial feature hierarchically in a layerwise training method, the input image patch should be flattened as a vector to meet the input requirement that may break the inherent spectral–spatial structure in HSI. We recommend that the input patch stays as the data cube to remain as much original information as possible. Lee and Kwon [30] incorporated residual learning with convolutional layers to form a contextual CNN, but spectral features and spatial features are processed in the same way that is intuitively incorrect. Meanwhile, the classification accuracies of the CNN models decrease when the networks become deeper. The spectral–spatial residual network (SSRN) [31] is developed to overcome this drawback. SSRN contains shortcut connections between every other convolutional layers; thus, a robust representation can be learned from the original HSI for classification. According to the previous research, we try to design a new architecture to extract better spectral–spatial feature and improve the HSI classification performance.

Although many deep learning-based methods have been introduced to enhance the classification performance, HSI classification still suffers from the following three problems.

1) The spectral–spatial information in HSI should be fully utilized in CNN to improve the classification performance. A proper way to organize the spatial and spectral information is needed.
2) The classification accuracy of the CNN models decreases when the network becomes deeper. A more effective network with fewer layers is needed.
3) The training procedure of CNN models is time consuming, and the training samples usually go hundreds of training sessions to converge.

All these problems limit the promotion of deep learning in HSI classification.

To counter these drawbacks, we propose a CNN with mapping layers (MCNN) in this article. As other deep networks extract the spectral–spatial features through multiple-layer mappings, the weights in all these layers are randomly initialized and updated by backward propagation. This leads to the problem that layers of network should go deep to extract the complicated and abstract features. Thus, an effective method to extract features of HSI may work better than the convolutional layers, especially when there is a large amount of spectral and spatial redundancies existing in HSI. By properly streamlining the redundancy and keeping the most significant part of energy in HSI, we can obtain effective HSI features. Despite the fact that deep convolutional layers can extract spectral–spatial features effectively, they also bring the problems of large computational cost and the classification accuracy decreasing with the network going deeper. In this article, we build a new kind of layer called mapping layer to effectively extract the feature and reduce the redundancy.

The mapping layer can reduce the redundancy in a specific mode of HSI tensor and keep the feature of the most part of the energy. The input of this kind of layer is the patch of HSI, and the data still retain the cube form. The spectral and spatial structures are kept, which are also beneficial to feature extraction. The weights of mapping layer are not updated by backward propagation; instead, they are obtained by decomposition of input HSI patches. This weight setting method can obtain more suitable weights for input HSI than randomly initialized weights in convolutional layers. The weights in the mapping layer are nontrainable; thus, the mapping layer can save much more computational cost than the convolutional layer because the weights in convolutional layers are updated by backward propagation in each training session. We adopt mapping layers to obtain better spectral–spatial feature and reduce the number of convolutional layers. Meanwhile, the problem that classification accuracy decreases with the network going deeper is avoided as our network architecture only has a few layers. Considering the difference between HSI and natural image is that HSI has much more bands, the bands should not be the redundancy but be the key to improve HSI classification performance. To extract the finer spectral–spatial feature, a 3-D convolutional kernel is also adopted. The 3-D convolutional kernel can extract the differences between bands as features, and more spectral information can help us improve the classification performance of our network.

The highlights of this article are listed in the following.

1) We build mapping layers to extract the spectral–spatial feature in HSI. We obtain the mapping kernels by decomposition of the input HSI patches, and the combination of these specialized layers for HSI with convolutional layers can extract better features for classification.
2) We propose a new network architecture called MCNN. Our MCNN contains three sections: mapping section, convolutional section, and fully connected section. The architecture takes into account feature extraction and dimension reduction; deploying only a few layers in MCNN architecture can save the computational time and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS ABOUT TENSOR

Notations used in this paper	
$\otimes$	Kronecker product
$\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$	A N_{th}-order tensor
$x(i_1, i_2, ..., i_N)$	The entry of tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$
$\mathbf{X}_{(1)}$	The mode-1 matricization of $\boldsymbol{\mathcal{X}}$
$\mathbf{X}^{I \times JK}$	A matrix of size $I \times JK$
$\mathbf{U}^{(i)}$	i_{th}-order
$\mathbf{X}_{(i)}^{k}$	The k_{th} slice along mode-i in tensor $\boldsymbol{\mathcal{X}}$
$\mathbf{X}_{::i}$	The i_{th} slice in tensor $\boldsymbol{\mathcal{X}}$

avoid the problem that accuracy decreases as network going deeper.

3) The Finer 3-D feature extraction method is adopted in convolutional layers. To extract the spectral–spatial feature, the input HSI patch is convoluted with 3-D kernels. As a result, we can obtain spatial and spectral features at the same time.

The rest of this article is organized as follows. Section II describes the related knowledge of tensor decomposition. We give our MCNN architecture in Section III. Then, the network configuration and experimental results are reported in Section IV. Discussions about the proposed mapping layer and performance are also offered in Section IV. Finally, conclusions are drawn in Section V.

II. TENSOR PRELIMINARIES

Tensor decomposition is a method that decomposes a tensor into one core tensor and factor matrices (projection matrices). The core tensor implies the connections among the vectors of factor matrices in different modes [32]. Two kinds of tensor decompositions are widely used: the canonical polyadic (CP) decomposition, also known as parallel factor (PARAFAC) [33] decomposition, and the Tucker decomposition (TD) [34], also known as high-order singular value decomposition (HOSVD) [35]. HSI is a data cube constructed by plenty of bands; therefore, it is naturally a third-order tensor. We can apply TD to decompose the tensor into three factor matrices and a core tensor where most of the energy is concentrated, and then, we can use this property to design our mapping layer and obtain better the spectral–spatial feature.

We list the notations used in this article in Table I. The order of the tensor is also known as mode. The first-order tensor is actually a vector, and the second-order tensor is known as matrix. The mode-n matricization means flattening the tensor into a matrix by fixing the Nth-order index and varying other indices. For a third-order tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I \times J \times K}$, there are three mode-n matricizations

$$\mathbf{X}_{(1)} = \mathbf{X}^{(I \times JK)} = [\mathbf{X}_{::1}, \ldots, \mathbf{X}_{::K}] \tag{1}$$

$$\mathbf{X}_{(2)} = \mathbf{X}^{(J \times KI)} = [\mathbf{X}_{:1:}, \ldots, \mathbf{X}_{:J:}] \tag{2}$$

$$\mathbf{X}_{(3)} = \mathbf{X}^{(K \times IJ)} = [\mathbf{X}_{1::}, \ldots, \mathbf{X}_{I::}]. \tag{3}$$

The mode-1 Kiers matricization of a third-order tensor $\boldsymbol{\mathcal{X}}$ results in $\mathbf{X}_{(1)}$, and the flattening process is shown in Fig. 1. $\mathbf{X}_{(2)}$ and $\mathbf{X}_{(3)}$ are the mode-2 and mode-3 Kiers matricization results, respectively. The matricizations in this article are all Kiers matricizations. The mode-n tensor-matrix product of a tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ multiplied by matrix $\mathbf{U} \in \mathbb{R}^{J_n \times I_n}$ is denoted as $\boldsymbol{\mathcal{X}} \times_n \mathbf{U}$

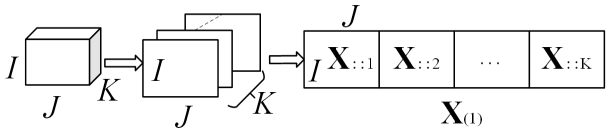


Fig. 1. Mode-1 matricization is obtained by flattening the tensor along its first mode.

$$\boldsymbol{\mathcal{X}} \times_n \mathbf{U}_{i_1, i_2, \ldots, i_{n-1}, i_n, i_{n+1}, \ldots, i_N} = \sum_{i_n=1}^{I_n} x_{i_1 i_2 \ldots i_N} \times u_{i_n j_n} \tag{4}$$

where the height of matrix $\mathbf{U}$ should be the same as the mode-n size of tensor $\boldsymbol{\mathcal{X}}$. This multiplication changes the mode-n size of tensor $\boldsymbol{\mathcal{X}}$ from I_n to J_n.

The Nth-order tensor can be resized to a new tensor with each mode resized as the height of the corresponding matrix. The equation is given as follows:

$$\boldsymbol{\mathcal{A}} = \boldsymbol{\mathcal{X}} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \cdots \times_N \mathbf{U}^{(N)} \tag{5}$$

where the width of matrix $\mathbf{U}^{(n)} (n = 1, 2, \ldots, N)$ should be equal to the size of mode-n of tensor $\boldsymbol{\mathcal{X}}$.

As the tensors in this article are all third-order tensors, TD is introduced in the version of third-order

$$\boldsymbol{\mathcal{X}} = \boldsymbol{\mathcal{G}} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \times_3 \mathbf{U}^{(3)} \tag{6}$$

where $\boldsymbol{\mathcal{X}}$ is the original tensor, $\boldsymbol{\mathcal{G}}$ is the core tensor of $\boldsymbol{\mathcal{X}}$, and $\mathbf{U}^{(1)}$, $\mathbf{U}^{(2)}$, and $\mathbf{U}^{(3)}$ are factor matrices (projection matrices), where $\mathbf{U}^{(n)} \mathbf{U}^{(n)T} = I_{J_n} (n = 1, 2, 3)$. The original tensor $\boldsymbol{\mathcal{X}}$ can be reconstructed by the following equation:

$$\boldsymbol{\mathcal{G}} = \boldsymbol{\mathcal{X}} \times_1 \mathbf{U}^{(1)T} \times_2 \mathbf{U}^{(2)T} \times_3 \mathbf{U}^{(3)T}. \tag{7}$$

III. PROPOSED MCNN ARCHITECTURE

CNN is an efficient method for classification and feature extraction. The application of CNN on the task of HSI classification significantly improves the classification accuracy. However, it still suffers from the problems that the classification accuracy of the CNN model decreases when the network becomes deeper [31], and the training time is long for the network with plenty of convolutional layers. In fact, all these drawbacks are caused by the increasing number of convolutional layers. The convolutional layers are adopted to extract spectral–spatial feature. Practically, two sequential 3×3 convolutional layers can perform better than a 5×5 convolutional layer [36]. Therefore, many researchers adopt more and more convolutional layers in their networks, which leads to the problems of decreasing classification accuracies and increasing computational time. Motivated by the above-mentioned analysis, we design a kind of more effective layer for HSI in this article, aiming at reducing the convolutional layer and extracting better spectral–spatial feature. We will first introduce our mapping layer and then give our

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

architecture based on mapping layers and convolutional layers. In the end, we will talk about using 3-D convolutional kernels to extract spectral–spatial features.

A. Mapping Layer

The input patch of HSI has spectral and spatial redundancies. In order to eliminate the redundancy and keep as much useful information for classification as possible, a special kind of layer for feature extraction is necessary. However, the convolutional layer only performs well on edge detection and texture recognition. Therefore, we newly design the mapping layer to extract the spectral–spatial feature and reduce the dimension.

For a third-order tensor, we need three mapping layers to extract energy concentrated features, and each mapping layer maps a corresponding mode (order). To generate adaptive mapping layers for all input HSI patches, we apply our method on the averaged patch of all training samples. In this way, our mapping layer can reduce the common redundancy of all training patches and save the training time

$$\underset{\mathbf{U}^{(1)},\mathbf{U}^{(2)},\mathbf{U}^{(3)}}{\arg\min} \quad \|\boldsymbol{\mathcal{X}} - f(\mathbf{U}^{(1)},\mathbf{U}^{(2)},\mathbf{U}^{(3)})\|_2^2 \tag{8}$$

where $f(\mathbf{U}^{(1)},\mathbf{U}^{(2)},\mathbf{U}^{(3)}) = \boldsymbol{\mathcal{G}} \times {}_1\mathbf{U}^{(1)} \times {}_2\mathbf{U}^{(2)} \times {}_3\mathbf{U}^{(3)}$. Obtaining the three factor matrices is a coupling problem described by (8), where $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1\times I_2\times I_3}$ is the input patch and $\mathbf{U}^{(n)} \in \mathbb{R}^{I_n\times R_n}$ is the factor matrix. R_n is the reduced dimensionality of the factor matrix, which can reduce the redundant dimension of the mode according to (4). A common strategy to optimize such a problem is the alternating least squares (ALS) method. We can solve only one factor matrix when other variables are fixed in the iteration. The optimal factor matrices are obtained after plenty of iterations.

We first initialize three factor matrices as $\{\mathbf{U}_0^{(1)}, \mathbf{U}_0^{(2)}$, and $\mathbf{U}_0^{(3)}\}$. The column vectors in initial factor matrices are orthogonal to each other. Then, the averaged HSI patch is flattened along three modes, and three corresponding matricizations $\mathbf{X}_{(1)}, \mathbf{X}_{(2)}$, and $\mathbf{X}_{(3)}$ are obtained. We update the factor matrices according to the following equations:

$$[\mathbf{U}^{(1)}, \mathbf{V}_1, \mathbf{T}_1] = \text{SVD}[\mathbf{X}_{(1)}(\mathbf{U}^{(3)} \otimes \mathbf{U}^{(2)}), R_1] \tag{9}$$

$$[\mathbf{U}^{(2)}, \mathbf{V}_2, \mathbf{T}_2] = \text{SVD}[\mathbf{X}_{(2)}(\mathbf{U}^{(1)} \otimes \mathbf{U}^{(3)}), R_2] \tag{10}$$

$$[\mathbf{U}^{(3)}, \mathbf{V}_3, \mathbf{T}_3] = \text{SVD}[\mathbf{X}_{(3)}(\mathbf{U}^{(2)} \otimes \mathbf{U}^{(1)}), R_3]. \tag{11}$$

In each iteration, three factor matrices are updated alternatively. The final factor matrices are obtained when all of the factor matrices can hardly be updated. These three factor matrices are used to construct three sequential mapping layers.

We use $\mathbf{U}^{(1)}, \mathbf{U}^{(2)}$, and $\mathbf{U}^{(3)}$ as the mapping kernel of three mapping layers correspondingly. The input of mapping layer is multiplied by mapping kernel according to (4). The input patch first goes through mapping layers. For mapping layer 1, the patch is mapped to a new tensor with its first dimensionality reduced to R_1. It means that the original tensor can be looked as $I_2 \times I_3$ vectors with I_1 features, and the mapping layer maps these $I_2 \times I_3$ vectors into shorter vectors with only R_1 energy concentrated features. The mapping layer 2 and mapping layer 3 are sequentially connected, and each mapping layer applies corresponding mode multiplication by (4). The procedures are also shown in Fig. 2. The input of mapping layer 1 is the HSI patch of size $I_1 \times I_2 \times I_3$, and the output of mapping layer 3 is a tensor of size $R_1 \times R_2 \times R_3$. Our mapping layers are used to extract the energy concentrated feature so that most energy of the input patch is still kept, which is because the weights in mapping kernel are actually the singular vectors of matricization.

Algorithm 1: Training of Mapping Layers

Input: The training patches $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1\times I_2\times I_3}$ and reduced dimensionalities R_1, R_2, R_3.

Initialized the factor matrix $\{\mathbf{U}_0^{(1)} \in \mathbb{R}^{I_1\times R_1}, \mathbf{U}_0^{(2)} \in \mathbb{R}^{I_2\times R_2}, \mathbf{U}_0^{(3)} \in \mathbb{R}^{I_3\times R_3}\}$;

Obtain $\overline{\boldsymbol{\mathcal{X}}}$ by averaging all training patches;

Obtain $\boldsymbol{\mathcal{G}}^0$ by (7);

Flatten $\overline{\boldsymbol{\mathcal{X}}}$ along each mode and obtain three matricizations $\mathbf{X}_{(1)}, \mathbf{X}_{(2)}, \mathbf{X}_{(3)}$;

while $\|\boldsymbol{\mathcal{G}}^{t-1} - \boldsymbol{\mathcal{G}}^t\|_F > 0.01$, **do**

  Obtain $\mathbf{U}^{(1),t}$ by (9);

  Obtain $\mathbf{U}^{(2),t}$ by (10);

  Obtain $\mathbf{U}^{(3),t}$ by (11);

  Obtain $\boldsymbol{\mathcal{G}}^t$ by (7), $t = t + 1$;

end

Output: The final $\mathbf{U}^{(1),t}, \mathbf{U}^{(2),t}, \mathbf{U}^{(3),t}$ are output as $\mathbf{U}^{(1)}, \mathbf{U}^{(2)}, \mathbf{U}^{(3)}$.

B. Architecture of MCNN

Since we apply three mapping layers in our network, the output of the mapping layer 3 is an energy concentrated tensor, while the spatial-spectral structure of the original patch is still reserved, which is beneficial for extracting the edge feature by convolution. As CNN has achieved human-level intelligence in several perception tasks [37], [38], we propose to use the convolutional layer to extract discriminative features in our method. However, convolutional layer can hardly reduce the feature dimension, which will affect the scale of weights in the following fully connected layers. Therefore, in order to reduce its output dimension, we connect each convolutional layer with a pooling layer. Moreover, it is quite effective to extract spectral–spatial feature from our energy concentrated tensor. Thus, two convolutional layers are enough to extract features and compose the features as discriminative information. In this article, we adopt max pooling in the pooling layers. Then, two fully connected layers are followed. The output of the last fully connected layer is a one-hot vector with length of class number.

The architecture of the proposed MCNN includes three sections, and the first section is called the mapping section, which is built for extracting the spectral–spatial feature. We take Indian Pines (IP) as an example to explain the proposed MCNN, the input of network is an HSI patch of size $13 \times 13 \times 200$, and three mapping layers have mapping kernels of sizes 13×7, 13×7, and 200×40, respectively. The input patch is mapped as a feature cube of size $7\times 7\times 40$. The energy in the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

HSI patch is concentrated, and most significant dimensions are reserved. The second section is a convolutional section, which includes two convolutional layers and two max pooling layers. This section is mainly built for extracting the edge features and composing them as abstract features for classification. The max pooling layer can reduce the feature dimension, and it usually follows the convolutional layer. The input of the first convolutional layer is $7 \times 7 \times 40$. We use a $5 \times 5 \times 10$ tensor as a convolutional kernel in this layer, and the stride of (1, 1, 5) is applied to subsampling the input. There are 64 kernels in this convolutional layer, and the output is a fourth-order tensor of size $5 \times 5 \times 54 \times 64$. A max pooling layer with pooling size of $3 \times 3 \times 5$ and stride of (1, 1, 2) is followed. We convolve the output of pooling layer with another 64 kernels of sizes $5 \times 5 \times 10$ as the second convolutional layer. A max pooling layer with the pooling size of $3 \times 3 \times 5$ and the stride of (1, 1, 2) is followed as well. These four layers construct the convolutional section. The fully connected section of two fully connected layers is mainly used to classify the obtained spectral–spatial features. Fig. 2 shows the architecture of our method.

The main difference between traditional CNN and our MCNN is the mapping layer. The main purposes of adopting these mapping layers are to more effectively extract the spectral–spatial feature and more efficiently reduce the redundancy. Therefore, multilinear algebra is applied in the mapping layer. We obtain our mapping kernels by optimizing the TD problem, and these kernels can well keep the spectral–spatial structure and reduce the unnecessary dimension.

C. 3-D Convolutional Kernel

Many CNN architectures have to deal with the large spectral redundancy, and the widely used PCA is applied to reduce the spectral dimension. The first three principal components (PCs) are usually extracted from HSI by PCA. However, these methods may lose plenty of spectral information. Considering this, we adopt the 3-D convolutional kernel, which is first developed by Chen *et al.* [27] to effectively extract the spectral–spatial feature in HSI.

The 3-D convolutional kernel performs convolution not only on the spatial direction but also on the spectral direction. The output of one kernel is a third-order tensor. The output at position (x, y, z) can be expressed as follows:

$$v_{x,y,z} = f\left(\sum_{0}^{T}\sum_{-P}^{P}\sum_{-Q}^{Q}\sum_{-R}^{R} w_{t,p,q,r}\alpha_{(x+p),(y+q),(z+r)} + b_{x,y,z}\right) \tag{12}$$

where (x, y, z) is the position in input patch, $(2P + 1)$, $(2Q + 1)$, and $(2R + 1)$ are the corresponding sizes of convolutional kernel, T is the number of channels (convolutional kernels), and w and b are weights in convolutional kernel. The weights w and b of 3-D convolutional kernel are all randomly initialized by the zero-mean Gaussian distribution. They are iteratively updated through the backpropagation procedure. $f(\cdot)$ is the nonlinear function to improve the nonlinear mapping capability. Since the output of the convolutional layer is a third-order tensor, the pooling layer is also a 3-D pooling layer, and the strides in convolutional layers and pooling layers are also in three directions.

We show the comparison of two kinds of convolutional kernels in Fig. 3. The advantage of the 3-D convolutional kernel is that it can extract spectral–spatial information simultaneously. For HSI classification, the structure information exists in the HSI patch, and 2-D convolution may not extract it completely. Thus, applying 3-D convolution is needed. Meanwhile, 3-D convolution involves fewer parameters than other deep learning-based methods on the same scale. Thus, it is more appropriate for problems with limited training samples, such as HSI classification. In this article, the 3-D convolutional kernel is of the size $5 \times 5 \times 10$.

IV. Experimental Results and Analysis

In this section, we introduce three commonly used HSI data sets, some experimental details, and results. We first validate the hyperparameters of our MCNN. Then, we compare our proposed MCNN with several state-of-the-art methods, and we also analyze the computational time of our network against other deep learning methods. Finally, we test the efficiency of our mapping layers. All of these experiments are tested on a computer of i7 7700 CPU @ 3.60 GHz and 24 GB of RAM and GeForce GTX1080Ti GPU.

A. HSI Data Sets

In this section, we first introduce three HSI data sets: IP, Pavia University, and Salinas. Then, we give our specified hyperparameters of our network, and we compare our network with several state-of-the-art HSI classification methods, including three deep learning methods, which are SSRN [31], deformable CNN (DHCNet) [39], and 3DCNN [27], [40], and two nondeep learning methods: edge preserve filtering (EPF) [15] and SVM. There are two 3DCNNs in our experiment: $3DCNN_1$ has the same convolutional section with our MCNN (only 2 convolutional layers), and $3DCNN_2$ has six convolutional layers. All these methods are evaluated by classification metrics: overall accuracy (OA), average accuracy (AA), and kappa coefficient (κ). In all these examinations, we take 20% of HSI patches as training samples; 10% of HSI patches are picked randomly from the rest HSI data set as a validation set, which do not include the training set, and the rest 70% of HSI patches are test samples.

The IP data set is collected by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) in Northwestern Indiana, which contains 16 vegetation classes. There are 145×145 pixels with a resolution of 20 m. There are 200 spectral bands in a range from 400 to 2500 nm, and 24 bands corrupted by water absorption effect and noise are discarded. The false-color composite of IP image and its ground truth map are shown in Fig. 4.

The University of Pavia (UP) data set is acquired by the Reflective Optics System Imaging Spectrometer in Northern Italy in 2001, which contains nine classes. There are 610×340 pixels with a resolution of 1.3 m/pixel. There are

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

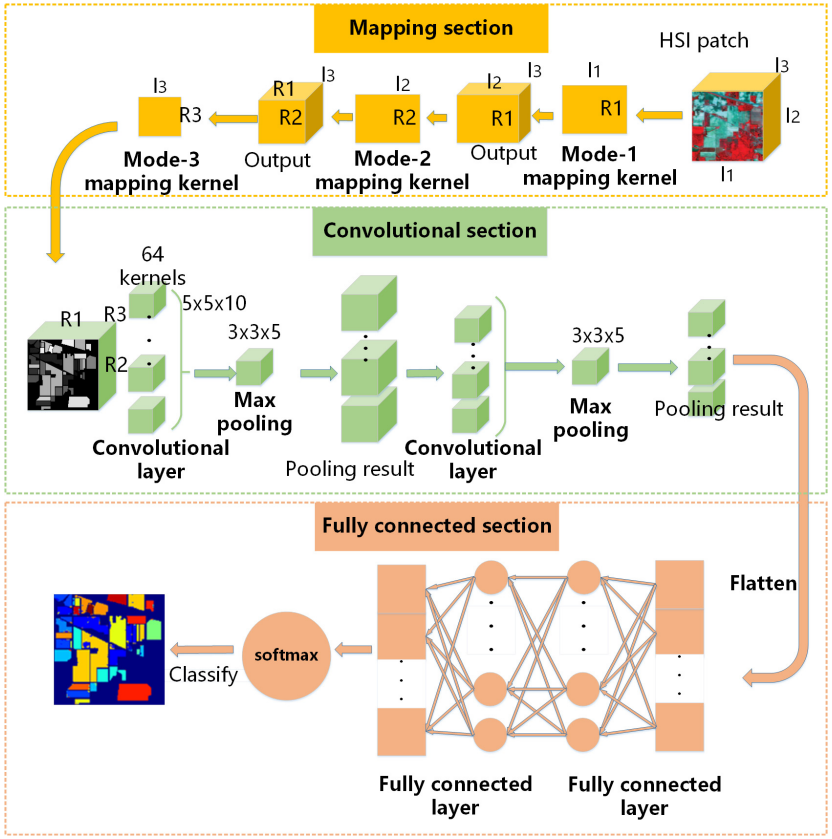


Fig. 2. Architecture of our proposed MCNN.

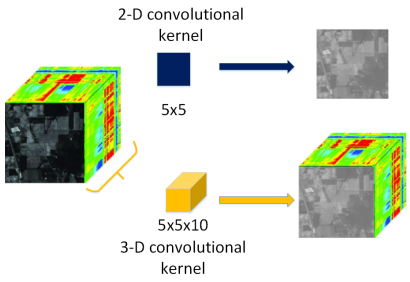
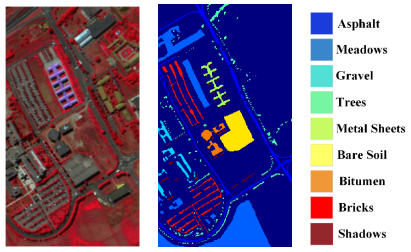
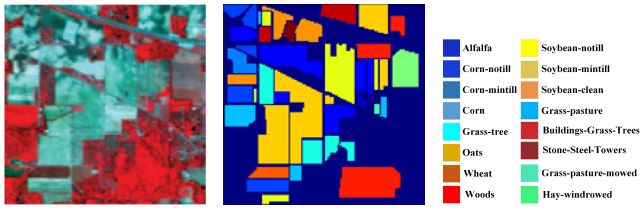


Fig. 3. Comparison of two kinds of convolutional kernels.



(a) (b) (c)

Fig. 5. (a) Three-band color composite of the UP image. (b) Ground truth of UP. (c) Color map.



(a) (b) (c)

Fig. 4. (a) Three-band color composite of the IP image. (b) Ground truth of IP. (c) Color map.

115 spectral bands in a range of 430–860 nm, and 12 most noisy bands are removed before experiment. The false-color composite of the UP image and its corresponding ground truth map are shown in Fig. 5.

The Salinas data set is collected by AVIRIS in Florida, which contains 16 classes. There are 512 × 217 pixels with a resolution of 3.7 m. There are 224 spectral bands in a range from 400 to 2500 nm, and all bands are reserved. The false-color composite of the Salinas image and its corresponding ground truth map are shown in Fig. 6.

The hyperparameters of comparative methods are set according to the published article. The sizes of input patch of $3DCNN_1$, $3DCNN_2$, SSRN, and our MCNN are $13 \times 13 \times$ band, where band is the number of spectral bands. Only DHCNet is $25 \times 25 \times$ band because this method requires larger spatial size. Due to those authors' efforts, SSRN,[1]

[1] https://github.com/zilongzhong/SSRN

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

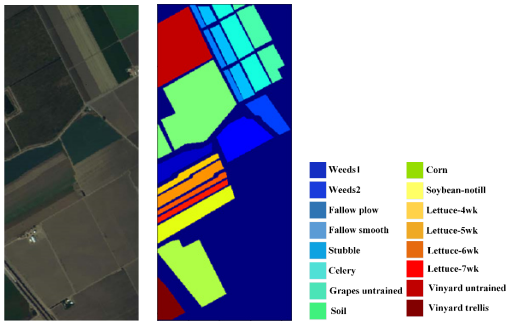


Fig. 6. (a) Three-band color composite of the Salinas image. (b) Ground truth of Salinas. (c) Color map.

DHCNet,[2] and EPF[3] are open sourced. The SVM algorithm is implemented in the LIBSVM[4] library [41]. Our code will be open sourced in Github.[5]

B. Validation of Hyperparameters

In this section, there are two kinds of hyperparameters needing validation: 1) the hyperparameters of convolutional networks, including learning rate, training epoch, and the batch size and (2) the hyperparameters of mapping layers, including output sizes of mapping layers: R_1, R_2, and R_3. We will first validate the hyperparameters of the convolutional network and then validate hyperparameters of our mapping layers based on the model of the validated hyperparameters. There are two reasons we can validate these two kinds of parameters separately. First, our mapping layers are a kind of preprocessing method for the input patch. The mapping layers are not updated by backpropagation; thus, they are not influenced by learning rate. Second, mapping layers extract spectral–spatial feature from each single input patch, which means that there is no need to validate the batch size with the hyperparameters of mapping layers.

For the convolutional blocks in our MCNN architecture, there are three hyperparameters that can control the training process and the classification performance. They are learning rate, training epoch, and the batch size in the convolutional layer. We use the Adam optimizer in our work, and R_1, R_2, and R_3 are specified as 7, 7, and 40 temporally for three data sets in accordance with our experience. The learning rate and batch size are validated together to find the best combination. Speci?cally, inappropriate learning rate settings will lead to divergence or slow convergence. Thus, we give ranges of batch size and learning rate as (20, 30, 40) and (0.01, 0.003, 0.001, 0.0003, 0.0001), respectively. The best combination of batch size and learning rate is searched among these two ranges. The result of the trained model is output every 10 epochs, and we take the epoch number of the best validation results as the validated epoch number.

[2]https://github.com/OrdianryCore/DHCNet
[3]http://xudongkang.weebly.com/
[4]https://www.csie.ntu.edu.tw/ cjlin/libsvm/
[5]https://github.com/dashaqi/MCNN

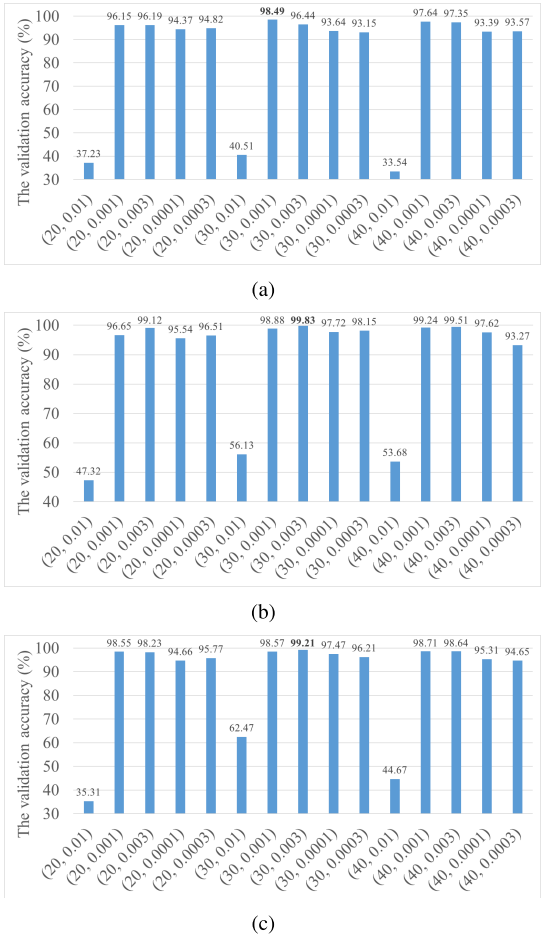


Fig. 7. Validation results of 15 models on three data sets, each model is the combination of batch size and learning rate. (a) IP. (b) UP. (c) Salinas.

All the validation accuracies in Fig. 7 are achieved on the best epoch number. It can be seen from Fig. 7(a) that the model with a batch size of 30 and a learning rate of 0.001 performs best on the validation set of IP. To find out the best epoch number of model (30, 0.001), we show the validation accuracy varying with epoch on the best model (30, 0.001) of IP in Fig. 8(a). When the model is trained 30 epochs, it can obtain the best performance on validation set. Thus, the most suitable hyperparameters of our MCNN on Inidan Pines are that the learning rate is specified as 0.001, the batch size specified as 30, and the training epoch specified as 30.

We also test the 15 models on UP and Salinas in the same way. The validation results are also given in Figs. 7(b) and (c) and 8(b) and (c). The results show that the best hyperparameters for UP and Salinas are the same. For UP and Salinas, we the set learning rate as 0.003, the batch size as 30, and the training epoch as 30.

As the hyperparameters of convolutional blocks are validated, then, we will validate the hyperparameters of R_1, R_2, and R_3 in our mapping layers. Since the independence of mapping layers, we can validate the hyperparameters of mapping layers and convolutional blocks separately.

The mapping layers are designed to extract the spectral–spatial feature. The input patch of our MCNN

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

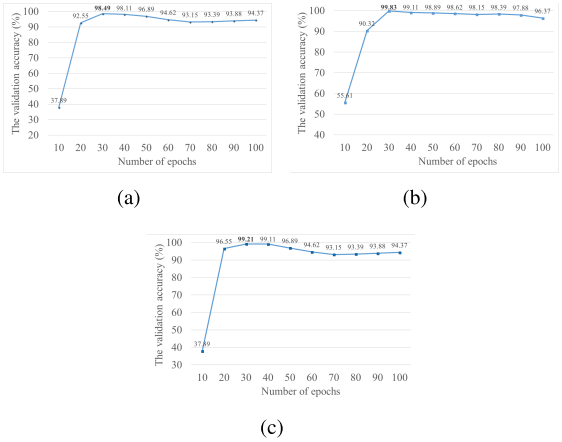


Fig. 8. Validation accuracy varies with the epoch number on the best model (batch size and learning rate). (a) Validation results of IP on the model (30, 0.001). (b) Validation results of UP on the model (30, 0.003). (c) Validation results of Salinas on the model (30, 0.003).

is a third-order tensor, and we use three mapping layers to extract the features of each order of the patch. Each mapping layer has a hyperparameter $R_n, n = 1, 2, 3$. These three hyperparameters can determine the sizes of extracted spectral–spatial feature, which can further determine the quality of features and computational consumption of MCNN. To find out the best combination of R_1, R_2, and R_3, we validate them on the validation data set. The hyperparameters of batch size and learning rate are set as (30, 0.001) for IP, and the hyperparameters of the other two data sets are set as (30, 0.003) and (30, 0.003) for UP and Salinas, respectively.

Considering our input is a third-order HSI patch, where R_1 and R_2 are spatial sizes and R_3 is the spectral size, we select the same value for R_1 and R_2. We validated R_1, R_2, and R_3 on three data sets: IP, UP, and Salinas. The validated combination of these hyperparameters and their classification accuracies are listed in the following.

The validation results show that the best hyperparameters of our mapping layers are different for three data sets. According to the results in Tables II–IV, we specify (R_1, R_2, R_3) as (7, 7, 40), (7, 7, 20), and (7, 7, 40) for IP, UP, and Salinas, respectively. The results also reveal that our mapping layers are able to extract better spectral–spatial features of small sizes, which further reduces computational cost and achieves better classification results.

C. Classification Performance

We test our methods with six comparative methods of SSRN, DHCNet, 3DCNN$_1$, 3DCNN$_2$, EPF, and SVM on IP, UP, and Salinas. Tables V–VII show the accuracy reports on three data sets. Figs. 9–11 show the classification maps obtained by different methods associated with the corresponding OA scores.

From Figs. 9–11, it can be seen that the classification obtained by SVM is not satisfactory since some noisy estimations are still visible. Among the deep learning methods, 3DCNN$_2$ performs better than 3DCNN$_1$ for the reason that the former has more convolutional layers than the latter.

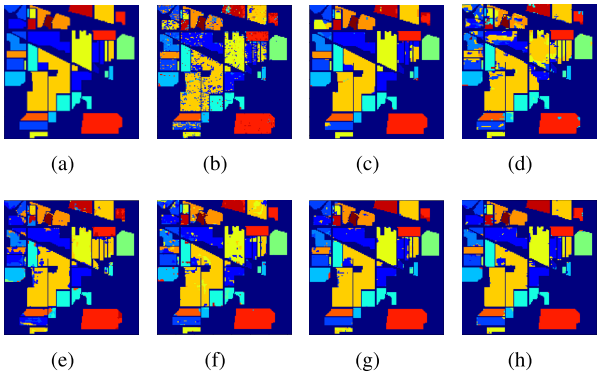


Fig. 9. Classification results of all methods for IP. (a) Ground truth. (b)–(h) Classification results of SVM, EPF, 3DCNN$_1$, 3DCNN$_2$, DHCNet, SSRN, and MCNN.

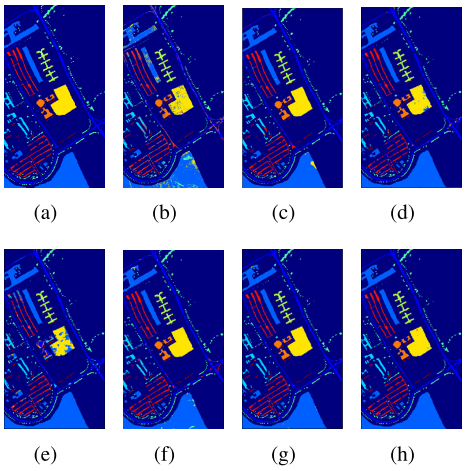


Fig. 10. Classification results of all methods for UP. (a) Ground truth. (b)–(h) Classification results of SVM, EPF, 3DCNN$_1$, 3DCNN$_2$, DHCNet, SSRN, and MCNN.

Although the classification performance decreases when more convolutional layers are deployed in CNN, two convolutional layers in 3DCNN$_1$ are clearly not enough to extract features from an input patch of sizes $13 \times 13 \times 200$. We specify the framework of 3DCNN$_1$ to compare with our MCNN as they both apply two convolutional layers. Our MCNN obtains the best OA scores, especially our MCNN increases the OA compared with the 3DCNN$_1$ method by about 2%–7%. These two methods have the same layers and architectures except our proposed mapping layers that verify the effectiveness of our mapping layers.

Our mapping layer can extract better spectral–spatial feature, and the feature concentrates the energy of the patch. Thus, mapping layers can reduce the redundancy and accelerate the following convolution options. As the mapping layer is designed for HSI (the kernel is not updated by backpropagation), it can obtain more effective features much faster than convolutional layers. Tables V–VII report the OAs, AAs, kappa coefficients, and the classification accuracies of all classes for HSI classification. In three cases, our MCNN achieves the highest classification accuracy.

We have compared the computational time of four neural network methods with our method in Table VIII–X. In these methods, DHCNet has employed PCA for data preprocessing,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

TABLE II
VALIDATION ACCURACY (%) VARIES WITH HYPERPARAMETERS R_1, R_2, AND R_3 ON IP

		R_3				
		20	40	60	100	140
R_1, R_2	11	95.05	98.06	95.14	96.56	95.60
	9	96.27	97.94	96.38	96.98	92.35
	7	93.51	**98.49**	97.18	96.39	94.56
	5	92.34	95.14	94.64	93.58	93.54

TABLE III
VALIDATION ACCURACY (%) VARIES WITH HYPERPARAMETERS R_1, R_2, AND R_3 ON UP

		R_3				
		20	30	40	50	60
R_1, R_2	11	99.01	97.06	99.64	99.65	99.68
	9	99.62	98.94	99.77	99.75	99.71
	7	**99.88**	99.12	99.83	99.73	99.60
	5	99.05	99.14	99.70	99.70	99.71

TABLE IV
VALIDATION ACCURACY (%) VARIES WITH HYPERPARAMETERS R_1, R_2, AND R_3 ON SALINAS

		R_3				
		20	40	60	100	140
R_1, R_2	11	97.46	98.51	98.44	98.64	97.68
	9	98.36	98.80	98.42	98.41	99.15
	7	98.40	**99.21**	98.45	96.39	98.89
	5	96.93	97.66	96.68	98.61	96.47

TABLE V
CLASSIFICATION ACCURACIES (%) OF OUR MCNN AND SIX COMPARATIVE METHODS TESTED ON IP

Class	SVM	EPF	$3DCNN_1$	$3DCNN_2$	DHCNet	SSRN	MCNN
Alfalfa	85.6 ± 1.2	98.3 ± 0.3	80.1 ± 1.2	95.4 ± 0.6	75.7 ± 0.3	97.7 ± 0.3	98.3 ± 0.7
Corn-N	80.1 ± 1.0	93.4 ± 0.2	75.5 ± 2.2	85.6 ± 0.5	92.3 ± 0.2	97.3 ± 2.2	96.2 ± 0.6
Corn-M	70.4 ± 2.5	97.3 ± 0.4	85.2 ± 1.2	89.2 ± 0.3	92.2 ± 1.5	99.1 ± 0.4	95.3 ± 1.8
Corn	80.1 ± 1.8	85.2 ± 2.8	86.3 ± 1.2	84.6 ± 0.7	85.6 ± 1.8	97.5 ± 1.7	94.2 ± 1.3
Grass-M	88.1 ± 1.1	98.4 ± 0.3	91.2 ± 0.8	94.3 ± 0.5	82.7 ± 2.8	97.2 ± 0.8	98.2 ± 0.2
Grass-T	94.4 ± 0.3	96.7 ± 0.2	98.1 ± 0.2	98.3 ± 0.2	92.3 ± 2.0	98.6 ± 1.2	98.7 ± 0.7
Grass-P-M	88.1 ± 1.1	95.4 ± 2.4	98.3 ± 0.7	80.2 ± 2.2	71.2 ± 3.2	95.5 ± 0.5	96.4 ± 1.3
Hay-W	98.5 ± 1.2	100 ± 0	96.2 ± 0.8	99.2 ± 0.2	86.3 ± 1.2	94.8 ± 0.8	100 ± 0
Oats	80.3 ± 3.2	100 ± 0	98.2 ± 1.1	100 ± 0	78.2 ± 4.3	98.6 ± 0.2	92.2 ± 1.7
Soybean-N	76.5 ± 3.8	88.2 ± 2.4	90.2 ± 1.8	85.2 ± 1.3	80.8 ± 3.7	99.5 ± 0.3	97.8 ± 0.5
Soybean-M	82.5 ± 1.5	92.2 ± 0.4	85.1 ± 1.2	87.2 ± 1.0	92.3 ± 2.2	95.8 ± 0.8	99.6 ± 0.2
Soybean-C	78.5 ± 2.6	94.6 ± 0.2	92.3 ± 0.4	83.8 ± 1.2	86.2 ± 4.2	92.1 ± 1.3	97.2 ± 1.2
Wheat	93.5 ± 0.3	99.6 ± 0.3	99.4 ± 0.2	98.9 ± 0.4	91.3 ± 2.4	100 ± 0	100 ± 0
Woods	94.2 ± 0.4	99.3 ± 0.1	98.2 ± 0.4	99.2 ± 0.3	97.2 ± 1.1	99.2 ± 0.2	99.7 ± 0.2
Buildings-G-T-D	75.2 ± 3.1	88.1 ± 2.4	93.1 ± 0.6	97.2 ± 1.2	86.3 ± 1.5	98.5 ± 0.6	98.8 ± 0.3
Stone-S-T	88.0 ± 2.5	89.0 ± 1.2	92.1 ± 0.8	92.5 ± 1.5	76.3 ± 4.5	98.6 ± 0.6	95.8 ± 1.5
OA	83.7 ± 0.3	94.3 ± 0.3	88.9 ± 0.3	90.7 ± 0.5	90.1 ± 1.1	97.4 ± 0.3	98.3 ± 0.2
AA	84.6 ± 0.4	94.7 ± 0.4	91.2 ± 0.4	91.9 ± 0.5	85.4 ± 0.6	97.5 ± 0.3	97.4 ± 0.4
$\kappa \times 100$	81.9 ± 0.3	93.6 ± 0.2	87.6 ± 0.5	89.6 ± 0.3	88.8 ± 0.7	97.1 ± 0.2	98.0 ± 0.3

and SSRN does not employ a data preprocessing method for the special architecture. The original method of SSRN aims at extracting spectral–spatial feature simultaneously. They use the HSI patch of size $13 \times 13 \times 200$ as input. The $3DCNN_1$ and $3DCNN_2$ also skip the data preprocessing step. Because of the same reason in SSRN, these two methods also emphasize the spectral–spatial feature extraction. To make the test fair, we can apply PCA on these methods in the same manner to reduce the spectral bands. If we employ PCA to reduce the feature dimension from $13\times13\times200$ to $7\times7\times40$, the spectral–spatial structure may be destroyed because a patch is flattened as a vector. Thus, we only use PCA to reduce the spectral bands to 40 for $3DCNN_1$, $3DCNN_2$, and SSRN.

Deep learning methods are time consuming. There are three points that may affect the running time: convolution operation, the number of convolutional layers, and the number of training epochs. We make improvements upon these three points. The input of our convolutional layer is smaller than that of the original HSI patch, which results in fewer convolution operations. Meanwhile, we design the mapping layer aiming at reducing the spectral and spatial redundancies in HSI. Therefore, less convolutional layers are needed in our MCNN.

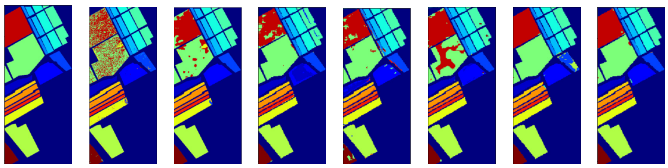
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CLASSIFICATION ACCURACIES (%) OF OUR MCNN AND SIX COMPARATIVE METHODS TESTED ON UP

Class	SVM	EPF	$3DCNN_1$	$3DCNN_2$	DHCNet	SSRN	MCNN
Asphalt	98.0 ± 1.3	99.0 ± 0.1	92.2 ± 1.2	98.2 ± 1.0	93.4 ± 0.6	99.1 ± 0.4	99.6 ± 0.3
Meadows	98.7 ± 0.2	99.6 ± 0.1	98.5 ± 0.1	97.2 ± 1.2	98.3 ± 1.5	99.5 ± 0.6	99.4 ± 0.3
Gravel	80.1 ± 2.3	97.1 ± 1.2	98.7 ± 1.3	96.2 ± 0.8	95.5 ± 1.2	99.2 ± 0.2	99.4 ± 0.2
Trees	86.8 ± 0.2	99.6 ± 0.2	99.7 ± 0.2	91.5 ± 0.8	98.5 ± 0.4	99.5 ± 0.4	99.8 ± 0.1
Metal sheets	97.5 ± 0.4	99.3 ± 0.2	99.8 ± 0.1	99.6 ± 0.3	96.5 ± 1.1	100 ± 0	99.9 ± 0.1
Bare soil	85.2 ± 3.5	94.7 ± 1.8	99.8 ± 0.2	97.8 ± 0.4	97.5 ± 5.2	98.8 ± 0.8	99.8 ± 0.2
Bitumen	72.2 ± 4.3	100 ± 0	84.5 ± 0.8	96.4 ± 0.9	98.6 ± 0.6	99.3 ± 0.5	98.8 ± 0.5
Bricks	87.5 ± 1.2	93.8 ± 0.8	99.2 ± 0.5	87.8 ± 0.5	97.5 ± 1.2	98.5 ± 0.4	98.9 ± 0.2
shadows	99.7 ± 0.3	100 ± 0	99.4 ± 0.3	100 ± 0	98.2 ± 0.3	100 ± 0	98.4 ± 0.4
OA	93.5 ± 0.4	98.3 ± 0.3	97.5 ± 0.4	96.3 ± 0.6	97.2 ± 0.8	99.3 ± 0.2	99.5 ± 0.2
AA	89.5 ± 0.3	98.1 ± 0.2	96.9 ± 0.5	96.0 ± 0.6	97.1 ± 0.5	99.3 ± 0.2	99.3 ± 0.1
$\kappa \times 100$	91.4 ± 0.3	97.8 ± 0.3	96.7 ± 0.3	95.1 ± 0.5	96.3 ± 0.6	99.0 ± 0.1	99.3 ± 0.2

TABLE VII
CLASSIFICATION ACCURACIES (%) OF OUR MCNN AND SIX COMPARATIVE METHODS TESTED ON SALINAS

Class	SVM	EPF	$3DCNN_1$	$3DCNN_2$	DHCNet	SSRN	MCNN
Weeds_1	97.6 ± 1.0	100 ± 0	80.2 ± 4.2	100 ± 0	95.8 ± 0.2	99.7 ± 0.1	100 ± 0
Weeds_2	99.2 ± 0.5	100 ± 0	88.2 ± 2.5	84.1 ± 2.1	99.6 ± 0.2	99.3 ± 0.1	99.4 ± 0.1
Fallow	91.2 ± 1.1	95.8 ± 0.2	99.2 ± 0.1	92.8 ± 0.5	99.8 ± 0.2	99.4 ± 0.2	99.5 ± 0.1
Fallow_P	99.1 ± 0.2	99.5 ± 0.1	99.3 ± 0.2	97.4 ± 0.2	96.0 ± 0.5	98.6 ± 0.2	99.4 ± 0.5
Fallow_S	95.8 ± 1.1	99.4 ± 0.3	99.5 ± 0.5	98.3 ± 0.8	98.7 ± 1.0	100 ± 0	98.2 ± 0.2
Stubble	100 ± 0	100 ± 0	99.5 ± 0.2	98.7 ± 0.5	99.3 ± 0.7	98.6 ± 0.7	99.7 ± 0.1
Celery	99.1 ± 0.1	95.4 ± 4.4	97.3 ± 0.7	97.2 ± 0.2	99.6 ± 0.2	99.0 ± 0.5	98.8 ± 0.3
Grapes	78.5 ± 4.5	90.8 ± 0.4	90.2 ± 0.3	91.2 ± 0.7	96.3 ± 1.2	99.6 ± 0.1	99.4 ± 0.2
Soil	99.2 ± 0.2	99.3 ± 0.3	100 ± 0	100 ± 0	99.6 ± 0.3	92.0 ± 0.4	99.7 ± 0.1
Corn	86.8 ± 1.2	92.2 ± 0.4	96.2 ± 0.4	99.2 ± 0.1	88.8 ± 0.2	100 ± 0	100 ± 0
Lettuce_4wk	86.5 ± 1.2	97.2 ± 0.3	98.2 ± 1.5	99.6 ± 0.1	100 ± 0	100 ± 0	100 ± 0
Lettuce_5wk	98.2 ± 2.3	96.5 ± 3.4	92.3 ± 0.4	83.8 ± 1.2	86.2 ± 4.2	92.1 ± 1.3	97.2 ± 1.2
Lettuce_6wk	93.2 ± 1.3	98.4 ± 1.2	98.5 ± 1.6	82.9 ± 4.4	98.1 ± 1.4	99.2 ± 0.2	99.7 ± 0.2
Lettuce_7wk	87.5 ± 0.4	99.1 ± 0.1	78.4 ± 3.4	87.2 ± 3.3	98.2 ± 1.1	99.2 ± 0.2	99.7 ± 0.2
Vinyard_U	75.8 ± 5.6	85.1 ± 2.4	98.1 ± 1.6	93.5 ± 0.2	83.3 ± 1.5	99.2 ± 0.2	98.8 ± 0.1
Vinyard_T	98.0 ± 0.3	95.4 ± 1.2	99.1 ± 0.5	99.5 ± 0.1	99.6 ± 0.2	98.2 ± 0.2	99.8 ± 0.1
OA	89.9 ± 0.6	94.7 ± 0.4	94.9 ± 0.5	94.3 ± 0.4	95.3 ± 0.3	98.3 ± 0.2	99.3 ± 0.2
AA	92.8 ± 0.5	96.5 ± 0.3	94.6 ± 0.6	94.1 ± 0.3	96.2 ± 0.4	98.4 ± 0.2	99.3 ± 0.1
$\kappa \times 100$	88.9 ± 0.8	94.1 ± 0.4	94.3 ± 0.4	93.7 ± 0.5	94.7 ± 0.3	98.0 ± 0.3	99.2 ± 0.2



(a) (b) (c) (d) (e) (f) (g) (h)

Fig. 11. Classification results of all methods for Salinas. (a) Ground truth. (b)–(h) Classification results of SVM, EPF, $3DCNN_1$, $3DCNN_2$, DHCNet, SSRN, and MCNN.

Our effective architecture can also save the number of training epochs. Since the training loss in our network decreases fast, merely 30 epochs or less are sufficient. In summary, the time of convolutional operations in one convolutional layer is saved, the number of convolution layers is reduced, and the training process is more effective in our MCNN architecture.

D. Efficiency of Our Mapping Layers

To further verify the advantage that our mapping layer can extract better spectral–spatial feature, we test our MCNN against three specified situations.

1) Mapping layers in our architecture are replaced by PCA transform. The PCA is employed as a preprocessing method on training data set and testing data set separately. The spectral bands are reduced to 40.

TABLE VIII
TRAINING TIME (S) ON IP BY DEEP LEARNING METHODS: SSRN, DHCNET, $3DCNN_1$, $3DCNN_2$, AND MCNN

Indian Pines	$3DCNN_1$	$3DCNN_2$	SSRN	DHCNet	MCNN
Training time for each epoch (s)	2.08	2.61	**0.74**	-	0.77
Epochs	30	30	200	-	30
Preprocessing time (s)	55.80	56.12	55.52	30.04	**0.26**
Training time in total (s)	118.24	134.52	203.47	393.59	**23.15**

2) Mapping layers are removed, and the 3-D convolutional layers are directly applied to classify HSI raw data. The input of MCNN is the original HSI patches.
3) Mapping layers are replaced by TD. We apply TD on each training patch and testing patch. The sizes of core tensor are the same as the output of our last mapping layer.

We test three situations on IP. The OA, AA, kappa coefficient, preprocessing time, and total training time results are listed in Table XI.

It can be seen that our proposed method performs best. In Situation 1, we replace our mapping layers with PCA, and its classification accuracy results are nearly 3% lower

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
TRAINING TIME (S) ON UP BY DEEP LEARNING METHODS: SSRN, DHCNET, $3DCNN_1$, $3DCNN_2$, AND MCNN

University of Pavia	$3DCNN_1$	$3DCNN_2$	SSRN	DHCNet	MCNN
Training time for each epoch (s)	5.81	8.03	**1.89**	-	3.34
Epochs	30	30	200	-	30
Preprocessing time (s)	72.34	73.46	73.16	56.25	**0.14**
Training time in total (s)	246.87	314.26	452.63	389.15	**100.21**

TABLE X
TRAINING TIME (S) ON SALINAS BY DEEP LEARNING METHODS: SSRN, DHCNET, $3DCNN_1$, $3DCNN_2$, AND MCNN

Salinas	$3DCNN_1$	$3DCNN_2$	SSRN	DHCNet	MCNN
Training time for each epoch (s)	3.35	5.35	1.25	-	**1.04**
Epochs	30	30	200	-	30
Preprocessing time (s)	64.25	65.74	63.51	48.41	**0.22**
Training time in total (s)	164.86	226.47	314.28	416.24	**31.53**

TABLE XI
CLASSIFICATION PERFORMANCE OF THREE SITUATIONS COMPARED WITH OUR MAPPING LAYERS

Indian Pines	PCA	Raw data	TD	Mapping layers
OA(%)	95.65	88.86	96.34	98.53
AA(%)	94.24	87.46	95.87	97.09
κ	0.9532	0.8785	0.9612	0.9769
Preprocessing time (s)	55.80	-	68.34	0.26
Total time (s)	118.24	266.25	90.12	23.15

than our method. The classification accuracy results show that PCA cannot extract spectral–spatial features as well as our mapping layers. In Situation 2, the mapping layers are removed to test the feature extraction performance. The classification accuracy results of Situation 2 are the lowest among four compared methods. It is easy to understand that mapping layers can extract good spectral–spatial features for classification, which is obviously better than the original raw data. In Situation 3, mapping layers are replaced by TD. TD can well decompose the input patch; however, each input patch is decomposed separately to obtain core tensors. Thus, the common spectral–spatial features are not well extracted, and the classification accuracy is not competitive with ours. Besides, the training time is also larger than our method.

The preprocessing time of our mapping layers is negligible because our mapping layers are obtained from the averaged training patches. Since there are K patches of sizes $M \times N \times Z$, PCA is applied on a matrix of size $KMN \times Z$, which needs Z^2KMN multiplications. TD is applied on K tensors of size $M \times N \times Z$, which needs $(MZ + MN + ZN)MNZK$ multiplications, while our method is applied on a tensor of sizes $M \times N \times Z$, which only needs $(MN)^2Z + (ZN)^2M + (MZ)^2N$ multiplications. Usually, K is a large number in HSI processing, which is saved in our method. Thus, the training time of our mapping layers (which is also the preprocessing time in Table XI) is negligible.

In summary, our mapping layers can extract better spectral–spatial feature, which helps to improve the classification accuracy performance, and mapping layers can also save the training time of our MCNN architecture.

The disadvantage of our MCNN is that it only avoids the gradient vanishing problem, and this problem can only be solved by the residual network so far. Meanwhile, our MCNN cannot extract features of different scales. These two problems will be investigated in our future work.

V. CONCLUSION

In this article, we have presented a neural network architecture of MCNN for spectral–spatial feature extraction and HSI classification. The proposed MCNN architecture, which contains a mapping section designed for spectral and spatial redundancy reduction, convolutional section to extract abstract features, and fully connected section to classify the features, has alleviated the spectral–spatial feature extraction and avoided the decreasing accuracy phenomenon. It is worth noting that our proposed mapping layer uses multilinear algebra to map the input HSI patch into lower dimensional subspace and maintains the spectral and spatial structures. Meanwhile, most energy of the patch is preserved. Moreover, our MCNN architecture simplifies the spectral–spatial feature extraction and improves the quality of spectral–spatial feature. It not only avoids the problem that classification accuracy decreases with the increasing number of convolutional layers but also saves the training time of the network. Furthermore, we employ the 3-D convolutional kernel to extract the spectral–spatial feature in convolutional layers. Experiment results demonstrated that our proposed MCNN performs consistently with the highest OA for all three commonly used HSI data sets with various challenges.

REFERENCES

[1] M. J. Khan, H. S. Khan, A. Yousaf, K. Khurshid, and A. Abbas, "Modern trends in hyperspectral image analysis: A review," *IEEE Access*, vol. 6, pp. 14118–14129, 2018.
[2] M. Fan, X. Zhang, L. Du, L. Chen, and D. Tao, "Semi-supervised learning through label propagation on geodesics," *IEEE Trans. Cybern.*, vol. 48, no. 5, pp. 1486–1499, Jul. 2018.
[3] X. Kang, S. Li, L. Fang, M. Li, and J. A. Benediktsson, "Extended random walker-based classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 1, pp. 144–153, Jan. 2015.
[4] Q. Wang, J. Lin, and Y. Yuan, "Salient band selection for hyperspectral image classification via manifold ranking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 6, pp. 1279–1289, Jun. 2016.
[5] Q. Du and H. Yang, "Similarity-based unsupervised band selection for hyperspectral image analysis," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 564–568, Oct. 2008.
[6] Y. Yuan, J. Lin, and Q. Wang, "Hyperspectral image classification via multitask joint sparse representation and stepwise MRF optimization," *IEEE Trans. Cybern.*, vol. 46, no. 12, pp. 2966–2977, Dec. 2016.
[7] J. Jiang, J. Ma, C. Chen, Z. Wang, Z. Cai, and L. Wang, "Super PCA: A superpixelwise PCA approach for unsupervised feature extraction of hyper spectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4581–4593, Aug. 2018.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.

[8] S. Prasad and L. M. Bruce, "Limitations of principal components analysis for hyperspectral target recognition," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 625–629, Oct. 2008.
[9] M. A. Hossain, M. Pickering, and X. Jia, "Unsupervised feature extraction based on a mutual information measure for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2011, pp. 1720–1723.
[10] V. Laparra, J. Malo, and G. Camps-Valls, "Dimensionality reduction via regression in hyperspectral imagery," *IEEE J. Sel. Topics Signal Process.*, vol. 9, no. 6, pp. 1026–1036, Sep. 2015.
[11] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.
[12] Y. Zhai *et al.*, "A modified locality-preserving projection approach for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 8, pp. 1059–1063, Aug. 2016.
[13] W. K. Wong, "Discover latent discriminant information for dimensionality reduction: Non-negative Sparseness Preserving Embedding," *Pattern Recognit.*, vol. 45, no. 4, pp. 1511–1523, Apr. 2012.
[14] Q. Wang, X. He, and X. Li, "Locality and structure regularized low rank representation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 911–923, Feb. 2018.
[15] X. Kang, S. Li, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification with edge-preserving filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 5, pp. 2666–2677, May 2014.
[16] G. Camps-Valls and L. Bruzzone, "Kernel-based methods for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 6, pp. 1351–1362, Jun. 2005.
[17] J. A. Benediktsson, M. Pesaresi, and K. Amason, "Classification and feature extraction for remote sensing images from urban areas based on morphological transformations," *IEEE Trans. Geosci. Remote Sens.*, vol. 41, no. 9, pp. 1940–1949, Sep. 2003.
[18] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.
[19] F. Galasso, R. Cipolla, and B. Schiele, "Video segmentation with superpixels," in *Computer Vision—ACCV 2012* (Lecture Notes in Computer Science), vol. 7724. Berlin, Germany: Springer, 2012, pp. 760–774.
[20] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013.
[21] N. Kruger *et al.*, "Deep hierarchies in the primate visual cortex: What can we learn for computer vision?" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1847–1871, Aug. 2013.
[22] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.
[23] Y. Kong, X. Wang, and Y. Cheng, "Spectral–spatial feature extraction for HSI classification based on supervised hypergraph and sample expanded CNN," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 11, pp. 4128–4140, Nov. 2018.
[24] J. Feng *et al.*, "CNN-based multilayer spatial–spectral feature fusion and sample augmentation with local and nonlocal constraints for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 4, pp. 1299–1313, Apr. 2019.
[25] X. Xu, W. Li, Q. Ran, Q. Du, L. Gao, and B. Zhang, "Multisource remote sensing data classification based on convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 937–949, Feb. 2018.
[26] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1156–1167, Feb. 2019.
[27] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.
[28] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, no. 12, pp. 3371–3408, Dec. 2010.
[29] H. Lee, R. Grosse, R. Ranganath, and A. Y. Ng, "Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations," in *Proc. Int. Conf. Mach. Learn.*, Jun. 2009, pp. 609–616.
[30] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.
[31] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.
[32] Q. Zhang, L. T. Yang, Z. Chen, and P. Li, "High-order possibilistic c-means algorithms based on tensor decompositions for big data in IoT," *Inf. Fusion*, vol. 39, pp. 72–80, Jan. 2018.
[33] R. A. Harshman, "Foundations of the PARAFAC procedure: Models and conditions for an explanatory multimodal factor analysis," UCLA Work. Papers Phonetics., Univ. Microfilms, Ann Arbor, MI, USA, Tech. Rep. 10,085, 1970, vol. 16, p. 184.
[34] L. De Lathauwer, B. De Moor, and J. Vandewalle, "A multilinear singular value decomposition," *SIAM J. Matrix Anal. Appl.*, vol. 21, no. 4, pp. 1253–1278, 2000.
[35] L. R. Tucker, "Some mathematical notes on three-mode factor analysis," *Psychometrika*, vol. 31, no. 3, pp. 279–311, 1966.
[36] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," Sep. 2014, *arXiv:1409.1556*. [Online]. Available: https://arxiv.org/abs/1409.1556
[37] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, p. 436, 2015.
[38] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
[39] J. Zhu, L. Fang, and P. Ghamisi, "Deformable convolutional neural networks for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 8, pp. 1254–1258, Aug. 2018.
[40] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, 2017.
[41] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, Apr. 2011, Art. no. 27.



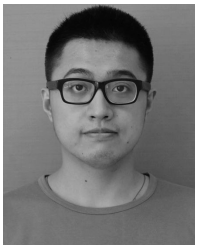
Rui Li received the B.S. and M.S. degrees from Xidian University, Xi'an, China, in 2012 and 2015, respectively. He is currently pursuing the Ph.D. degree with the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an.

His research interests include vector quantization and hyperspectral image processing.



Zhibin Pan received the B.S. degree in information and telecommunication engineering and the M.S. degree in automation science and technology from Xi'an Jiaotong University, Xi'an, China, in 1985 and 1988, respectively, and the Ph.D. degree in electrical engineering from Tohoku University, Sendai, Japan, in 2000.

He is currently a Professor with the Department of Information and Telecommunication Engineering, Xi'an Jiaotong University. His research interests include image compression, multimedia security, and object recognition.



Yang Wang received the B.S. and Ph.D. degrees from the School of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an, China.

He is currently a Lecturer with the Department of Information Science, Xi'an University of Technology, Xi'an. His research interests include image coding and image processing.



Ping Wang received the B.S., M.S., and Ph.D. degrees in information and communication engineering from Xi'an Jiaotong University, Xi'an, China, in 1999, 2002, and 2011, respectively.

She is currently an Associate Professor with the School of Electronic and Information Engineering, Xi'an Jiaotong University. Her research interests include image processing, video coding, and video analysis.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 09:06:03 UTC from IEEE Xplore. Restrictions apply.